ARGUS - Complete Project Overview

Everything about what we have built, how it works, and where it differs from the original plan. Source: the code in this repository (read in full), `README.md`, `docs/*.md`, and the original project plan PDF.

---

1. What is this project?

ARGUS (Adaptive Reasoning & Grounded Unified Security agent) is a conversational AI-style agent that runs an **authorized** vulnerability assessment and penetration test (VAPT) from start to finish:

> *request in plain English -> scope check -> plan -> approval -> real tool scans -> merge duplicates -> remove false positives -> rank risk -> human approval for risky steps -> client-ready PDF report*

It was built for **DataOps Innovation Hackathon 2026, Problem Statement 1** (Hackthecore, 2-7 Nov 2026, virtual on Discord, cybersecurity domain). Team: Rahul (RAHUL0408-B) and Pawan. Repo: github.com/RAHUL0408-B/DataOps.

The pitch: "Scanners give you alerts. ARGUS gives you conclusions: each one comes with the evidence behind it, a confidence level, and the reasoning that led to it. A human approves every risky action."

What makes it different from "run nmap -> nikto -> zap, then ask an LLM to summarise": - It runs an *investigation loop* - it looks at what it knows, forms hypotheses, picks the next cheapest permitted check, runs it, updates its beliefs. - Every step is written down as a **Decision Card** (what it saw, options considered, why it chose one). - Every finding is traceable to saved raw tool output with a SHA-256 hash. - Safety is enforced by plain code (a scope guard), not by asking an AI to behave.

---

2. Glossary - plain-English definitions of the security terms used in this doc

If cybersecurity is new to you, read this section first; everything below assumes these words.

The big picture - **VAPT** = **V**ulnerability **A**ssessment and **P**enetration **T**est. "Vulnerability assessment" means finding weaknesses (like a home inspector finding a broken lock). "Penetration test" means actually trying to use that weakness to get in (like a locksmith proving the lock can be picked). ARGUS does the finding automatically and the "trying it" part only with a human's explicit approval, and only with safe, read-only checks - never anything destructive. - **Target / asset** = a computer, server or web app being tested. In this project the targets are two deliberately-broken practice apps running in Docker (see section 6), never anything on the real internet. - **Scope** = the list of targets you are *allowed* to test, and which ones are off-limits. Testing anything outside scope without permission is illegal (unauthorized access), so ARGUS refuses by default and only allows what's explicitly listed. - **Finding** = one specific weakness ARGUS believes it found, e.g. "this login page is vulnerable to SQL injection." Each finding carries evidence, a confidence score and a fix. -

False positive = a finding that looks real but isn't (a scanner misread something). ARGUS tries to catch these with a second, independent check before trusting a finding.

Common weakness types you'll see in findings - **SQL injection (SQLi)** = a web form that passes your input straight into a database command. If the input isn't checked, an attacker can type database commands instead of normal data (e.g. a username) and read or change data they shouldn't see. One of the most common and serious web bugs. - **XSS (Cross-Site Scripting)** = a web page that echoes your input back into the page without cleaning it, letting an attacker inject their own script that runs in another user's browser (e.g. to steal their login session). "Reflected" XSS comes from a single malicious link; "stored" XSS is saved on the server and hits every visitor. - **Cleartext protocol** = data sent over the network unencrypted (plain HTTP instead of HTTPS, plain FTP, etc.), so anyone who can see the network traffic (e.g. on the same Wi-Fi) can read passwords or data in transit. - **Outdated software / version-based finding** = the scanner identified the exact software and version a server is running (from a banner it announces), and that version is publicly known to have bugs. This is a *weak* signal on its own, because version banners can be wrong or faked - which is why ARGUS treats these as "suspected" until something re-confirms them. - **CVE** (Common Vulnerabilities and Exposures) = a public ID number (like `CVE-2021-44228`, the Log4Shell bug) assigned to one specific, documented software vulnerability. Think of it as a case file number for a known bug. - **CVSS** (Common Vulnerability Scoring System) = a standard 0-10 severity score for a vulnerability, used to compare how bad different bugs are (10 = critical). - **CWE** (Common Weakness Enumeration) = a category ID for a general *type* of bug (e.g. CWE-89 is "SQL Injection" as a category), as opposed to CVE which is one specific instance of a bug in one specific product. - **KEV** (Known Exploited Vulnerabilities) = a government-maintained list (CISA's) of CVEs that are confirmed to be actively used by real attackers right now, not just theoretically dangerous.

Tools and techniques ARGUS runs - **nmap** = a tool that scans a target to find out which network ports are open and what software (and version) is listening on each one. This is always the first step - you can't test a service you don't know exists. - **Nikto** = a tool that checks a web server for common misconfigurations and known-risky files (an older, simpler scanner than ZAP). - **OWASP ZAP** = a full web-application scanner. Its "passive scan" just reads traffic without attacking anything (safe, runs automatically). Its "active scan" actually sends attack-like payloads to see if they work (e.g. trying real SQL injection strings) - this can affect the app, so ARGUS always makes a human approve it first. - **Metasploit** = a well-known penetration-testing framework containing ready-made exploit and scanning modules. ARGUS only uses its safe, read-only "auxiliary" scanner modules (to double-check a version number, for example), from a fixed allow-list, and only after human approval - never an actual exploit module. - **DVWA** (Damn Vulnerable Web Application) and **OWASP Juice Shop** = two intentionally broken web apps built for security training. They are the practice targets in this project's Docker lab - safe to attack because they were made to be attacked. - **Spider / crawl** = having a tool follow every link on a website automatically to build a map of all its pages, so later scans know what to check. - **Passive vs. active vs. intrusive vs. exploit** = an escalating scale of how risky/aggressive a check is. Passive only reads/observes; active sends test payloads; intrusive can affect the app's behavior or state; exploit would actually take advantage of a bug. ARGUS requires explicit human approval before anything intrusive or above. - **Nessus / Burp Suite / OpenVAS** = well-known commercial (or semi-commercial) vulnerability scanners. ARGUS doesn't run these live itself (no license available for the hackathon); instead it can *import* a report file one of these tools already produced and fold those findings into its own analysis.

How ARGUS reasons about what it finds - **Hypothesis** = a guess ARGUS forms about what might be wrong (e.g. "this web server might have misconfigurations") before it has proof. It picks the cheapest, safest next check to test that guess. - **Confidence score** = a 0-100 number ARGUS assigns to each finding based on *how* it was found (seen by one tool vs. confirmed by two independent tools vs. actually re-checked and confirmed) - shown as Low/Medium/High. - **Status ladder** (observed -> suspected -> corroborated -> validated -> refuted -> fixed) = how sure ARGUS is about a finding over time: first noticed, then another signal supports it, then two tools agree, then it was independently re-checked and

confirmed true, or re-checked and shown to be wrong (refuted), or confirmed fixed on retest. - **Grounding** = making sure any text an AI model writes (an explanation, a rationale) only states facts ARGUS can actually point to in its own evidence - never something the AI invented. ARGUS checks this automatically and rejects/relabels anything it can't verify. - **Decision Card** = a written record of one decision ARGUS made: what it knew, what options it considered, and why it picked the one it ran. This is what makes the process auditable instead of a black box. - **Audit log / hash-chained** = a tamper-evident record of every action taken. "Hash-chained" means each entry contains a fingerprint of the previous entry, so if anyone edited an old entry, every entry after it would no longer match - making silent tampering detectable. - **Priority / risk score** = a single 0-10 number combining a finding's base severity with how exposed, exploitable and important the affected system is, so the report can be read top-to-bottom as "fix this first."

---

3. Input and output at a glance

Inputs

Input	Where it comes from	Example
Request text	Text box in the UI (or `POST /api/assessments`)	"Assess the lab network 10.77.0.0/24, focus on web applications. Passive and standard checks first; no denial of service."
Scope	Hard-coded lab scope (`lab_scope()` in `backend/argus/api.py`) or a `scope` object sent with the request	Targets `10.77.0.0/24`, exclusions `.1 .2 .3`, max intensity `intrusive`, authorized by "Team ARGUS"
Imported scanner files (optional)	UI upload or sample buttons	Nessus `.nessus`, Burp issues XML, OpenVAS report XML
Human decisions	Approvals tab	Approve plan; approve/reject ZAP active scan or Metasploit check, with analyst name and comment
Env vars (optional)	Environment	`GEMINI_API_KEY` or `ANTHROPIC_API_KEY` (turns on the AI planner; Gemini preferred if both set), `ARGUS_DATA`, `ARGUS_TOOL_TIMEOUT`, `ARGUS_MAX_STEPS` ...

Outputs

Output	Where
PDF VAPT report	`GET /api/assessments/{id}/report.pdf` (Report tab)
Ranked findings with evidence, confidence reasons, priority formula, remediation	Findings tab / API
Decision Cards (agent reasoning log)	Plan & reasoning tab, PDF Appendix C

Output	Where
Attack-path graph	Attack paths tab
Hash-chained audit log	Report & audit tab, `/api/assessments/{id}/audit`, PDF Appendix D
Raw evidence files with SHA-256	`data/artifacts/<assessment>/...`, viewable via UI/API
Live event stream	Server-sent events at `/api/assessments/{id}/events`

---

4. Tech stack

Layer	What we used
Backend language	Python 3.11+ (developed on 3.14)
Web framework	FastAPI + uvicorn (REST + SSE + serves the UI)
Data models	pydantic v2
Storage	SQLite (plain `sqlite3`), 3 tables: `objects` (JSON documents), `audit` (hash chain), `events` (SSE feed)
HTTP client	httpx (talks to ZAP's REST API)
PDF	reportlab (uses Windows Segoe UI/Consolas fonts if present, else Helvetica/Courier)
Frontend	Vanilla JavaScript single page (`frontend/app.js`, `index.html`, `style.css`, SVG illustrations). No build step. Hand-drawn SVG for charts and the attack graph
Lab	Docker Compose: DVWA, OWASP Juice Shop, a Debian "tools" container (nmap, nikto, curl), OWASP ZAP daemon
Security tools	nmap, Nikto, OWASP ZAP (live); Metasploit read-only modules (live, approval-gated); Nessus, Burp, OpenVAS (import-only)
Optional AI	Google Gemini (REST, `gemini-flash-lite-latest`) if `GEMINI_API_KEY` is set, else Anthropic Claude (`anthropic` SDK, `claude-sonnet-5-5`) if `ANTHROPIC_API_KEY` is set. Gemini verified live; Claude unit-tested only
Tests	pytest (34 unit tests) + a live end-to-end script (`scripts/e2e_live.py`, ~100 checks against the real lab)
Run scripts	PowerShell: `scripts/start.ps1`, `scripts/stop.ps1`
License	MIT (ARGUS code only; third-party tools keep their own licenses)

`requirements.txt` : fastapi, uvicorn[standard], pydantic, httpx, reportlab, anthropic (optional), pytest.

---

5. Repository layout

```
backend/argus/
  api.py            FastAPI app: all REST routes, SSE stream, static UI mount, default lab scop
  config.py         paths, env vars (tools container, ZAP URL, MSF image, timeouts, max steps)
  models.py         pydantic types: Scope, Evidence, Asset, Finding, Hypothesis, Candidate,
                    DecisionCard, Job, Approval, Assessment
  scope.py          deterministic scope & policy guard (default deny)
  store.py          SQLite persistence, hash-chained audit log, event stream
  runner.py         runs commands: timeout, 1 retry, cancel/kill switch, saves evidence + SHA-2
  correlation.py    normalize, merge duplicates, Confidence Ledger, validation application
  risk.py           transparent priority formula and ranking
  graph.py          attack-path graph (verified vs hypothesis edges)
  grounding.py      anti-hallucination validator (see caveat in section 12)
  reference.py      offline data: partial CISA KEV, version->CVE table, CWE/OWASP map, ATT&CK m
                    issue-class regexes + remediation text
  report.py         PDF generation
  agent/planner.py  candidate action generation, rule planner, optional Claude planner
  agent/engine.py   the assessment engine: parse request, plan, loop, approvals, import, retest
  adapters/         nmap.py nikto.py zap.py msf.py validators.py imports.py base.py __init__.py
backend/tests/      test_pipeline.py, test_robustness.py, test_scope_audit.py
frontend/           index.html, app.js, style.css, img/*.svg
lab/                docker-compose.yml, tools/Dockerfile
samples/            synthetic Nessus/Burp/OpenVAS files (written by the team, not real scanner
scripts/            start.ps1, stop.ps1, e2e_live.py, make_test_report.py
docs/               ARCHITECTURE.md, INTEGRATIONS.md, AI_DISCLOSURE.md, project plan PDF, sampl
```

---

6. The lab (what we test against)

`lab/docker-compose.yml` creates an isolated Docker network `argus-lab_labnet` (10.77.0.0/24):

IP	Container	Role	In scope?
10.77.0.10	argus-dvwa	deliberately vulnerable web app	**yes (target)**
10.77.0.11	argus-juice	OWASP Juice Shop	**yes (target)**
10.77.0.2	argus-tools	nmap, nikto, curl run here	excluded

IP	Container	Role	In scope?
10.77.0.3	argus-zap	ZAP daemon (API exposed on 127.0.0.1:18090)	excluded
10.77.0.1	Docker gateway = **your own PC**	-	**excluded**

The exclusion of `.1/.2/.3` exists because an early run reached the Docker gateway (the developer's PC). That was fixed and is a good story for the demo.

One-time lab setup. A fresh DVWA container has no database and ZAP has no authenticated session for it, so `scripts/start.ps1` runs `scripts/lab_init.ps1` automatically after `docker compose up`: it creates the DVWA DB, logs in as `admin` / `password`, confirms the security level is `low`, and registers that session with ZAP's `httpSessions` API so ZAP's own spider and active scan (which never fill in a login form on their own) ride an authenticated session and actually reach `/vulnerabilities/*` instead of bouncing off `login.php`. (A ZAP *Replacer* rule looked like the obvious tool for this and silently did nothing - it only replaces a Cookie header that already exists, never adds one. `httpSessions` is the API actually built for forcing cookies onto every request.) Safe to re-run any time the lab containers are rebuilt.

---

7. How it works - the full flow

Step 0: Create assessment (`engine.create_assessment`)

1. The text is parsed with regex: **targets** (IPs, CIDRs, URLs, hostnames) and **intents** (`no_web` , `web_only` , `passive_only` , `no_dos`).
2. Each requested target goes through `scope.check_target` . Accepted targets are kept; rejected ones are recorded as *limitations* and **no traffic is ever sent** to them.
3. If nothing is in scope, the assessment is marked `failed` ("nothing will be tested").
4. If no target is mentioned, the registered scope targets are used.
5. Everything is written to the audit chain and the event stream.

Step 1: Plan (`engine.make_plan`)

The planner creates candidate actions from current state (at this point only `nmap_scan:<target>`), picks one with `rule_choose` , and stores a **Decision Card**. Status becomes `planned` . **Nothing has run yet.** The analyst must press **Approve plan**.

Step 2: Investigation loop (`engine._loop` , max 24 steps)

Each iteration: 1. **Generate candidates** from state (`planner.generate`), plus hypotheses. 2. **Choose** one: `rule_choose` sorts runnable (non-approval) candidates by *phase order*, then *most hypotheses resolved*, then *lowest cost*. If `GEMINI_API_KEY` or `ANTHROPIC_API_KEY` is set, `llm_choose` asks that model (Gemini preferred) to pick among the same candidates instead; any invalid answer falls back to rules. 3. **Write a Decision Card**: trigger evidence, hypothesis, every option with CHOSEN/deferred/needs-approval/skipped, rationale, scope check result. 4. **Execute** (`engine.execute`): scope `enforce` -> audit `action.start` -> run adapter -> save assets -> correlate findings ->

audit `action.done`. Failures (tool unavailable, timeout, crash) become a **coverage gap** in the assessment's limitations; the loop continues. 5. **Update hypotheses** (open -> supported/refuted). 6. When no runnable action is left, the loop ends and `finalize` runs.

Typical order the rule planner produces: `nmap_scan` (discover hosts/services) -> `nmap_enum` (NSE http scripts on web ports) -> `nikto` and `zap_passive` per web service -> `validate` (batch of one-GET checks) -> done.

Candidate details (from `planner.generate`): - **Recon:** `nmap_scan:<target>` until some asset is known. - **Hypotheses:** one per web service ("has misconfigurations or known weaknesses"), one per CVE-suspected finding, one per cleartext-protocol finding. - **Enumeration / web scans** (skipped if request said "no web"; recorded as `skip-web:` with reason): `nmap_enum`, `nikto`, `zap_passive`. - **Validation:** one `validate:<finding>` per config finding whose class is in `VALIDATABLE` (clickjacking, missing X-Content-Type-Options, missing CSP, missing HSTS, server version disclosure, cookie HttpOnly/Secure/SameSite, directory listing). - **Approval-gated** (only proposed once findings exist and the request wasn't "passive only"): `zap_active` (if scope allows intrusive) and `msf_check` (for version-only CVE suspicions, if the Metasploit image is pulled).

Step 3: Finalize (`engine.finalize`)

1. `correlation.link_related` - cross-reference findings on the same asset/CVE.
2. `graph.build` - attack-path graph and path bonuses.
3. `risk.rank` - compute priorities, rank findings, mark refuted ones as excluded.
4. `_propose_approvals` - create `Approval` objects for intrusive candidates.
5. Status becomes `awaiting_approval` (if any pending) or `review`.

Step 4: Human approvals (`engine.decide`)

- Approve -> runs the action in a background thread with actor `<analyst>(approved)`, then re-finalizes.
- Reject -> recorded, not executed.
- A failed approved action can only be retried through a **new** approval (`request_again`) - each execution needs fresh consent.
- Analysts can also *request* a Metasploit check on any finding (`propose_validation`); the module must be on the allow-list and the target still passes scope.

Step 5: Import, retest, ask, report

- **Import** (`engine.import_file`): parse Nessus/Burp/OpenVAS file, drop findings whose host is out of scope (recorded as a limitation), merge into existing findings, write a Decision Card with planner `analyst`. Labelled IMPORT everywhere.
- **Retest** (`engine.retest`): re-run the single safe GET for a validatable finding. If the issue is gone -> status `fixed`; if still there -> `validated`. Failure to retest times out cleanly with HTTP 504 and leaves the finding unchanged.
- **Ask ARGUS** (`engine.ask`): rule-based, grounded Q&A over stored data: "why is F-003 ranked high?", "show evidence for F-002", "summary", "pending approvals". **No model is called.**
- **Report**: PDF built from stored data on demand.

Kill switch and crash recovery

- `POST /stop` cancels running subprocesses and sets a cancel flag; the loop raises `InterruptedError` and the assessment becomes `stopped`.
- On server start, `recover_interrupted` marks any assessment left in `running` as `stopped`, cancels its jobs and approved-but-unfinished approvals, and notes it in limitations.

---

8. Tool integrations (honest status)

Tool	Status	How it runs	Notes
nmap	Live	`docker exec argus-tools nmap -sV -T4 --top-ports 1000 --open -oX -` for discovery; `-p <ports> --script http-title,http-methods,http-server-header,http-enum` for enumeration. XML parsed. Excluded IPs passed via `--exclude`.	Rule findings: cleartext protocols (telnet/ftp/...), exposed DB ports, risky HTTP methods, version->CVE from a small reference table (Apache 2.4.49/2.4.50, vsftpd 2.3.4). No `vulners` script.
Nikto	Live	`docker exec argus-tools nikto ... -Format json`, output read back then deleted	Nikto gives no severity/CVSS; ARGUS derives a coarse "low/info" from the message and labels it *derived*.
OWASP ZAP	Live	ZAP daemon REST API (Host header `zap`): spider -> (active scan) -> wait for passive scan -> fetch up to 500 alerts + URLs	Active scan is restricted to SQLi + reflected/persistent-XSS scanners (the exact classes it exists to confirm) and capped at 4 min; if it overruns, ARGUS stops it and keeps partial alerts, labelled partial. No CVSS invented.
Metasploit	Live, approval-gated	`docker run --rm --network argus-lab_labnet metasploitframework/metasploit-framework ./msfconsole -q -x "use <module>; set RHOSTS..; run; exit"`	Only 6 read-only auxiliary modules (http_version, http_header, options, robots_txt, ftp_version, ssh_version). **No exploit modules, no payloads.** Never autonomous. Confirms a finding only if it's version-related (server-disclosure/outdated-software/CVE) AND the module output reproduces the version string; otherwise the verdict is inconclusive, never a false confirm.
Nessus	Import-only	Parses `.nessus` v2 XML	Info-level plugins are context, not findings. No API.
Burp	Import-only	Parses issues XML	Pro REST API is *Planned*, not implemented.
OpenVAS	Import-only	Parses report XML	Live GMP integration *Planned*.

Tool	Status	How it runs	Notes
AI planner (Gemini / Claude)	**Live (Gemini), untested (Claude)**	Ranks the same code-generated candidates, writes the rationale	Gemini (`gemini-flash-lite-latest` , free tier) is tried first if its key is set, else Claude. **Verified live**: ran a real assessment end-to-end with `GEMINI_API_KEY` set; the engine's decision log shows `"planner": "gemini"` and genuine contextual rationale (e.g. correctly reasoning about a discovered "Apache httpd 2.4.25" service, not generic text). Claude remains unit-tested only (no key available).
Exploit frameworks beyond read-only	Unsupported	-	By design.

The UI "Tools" tab (`GET /api/matrix`) probes each tool at runtime so the matrix never claims more than the running system can do.

---

9. Core engines in detail

8.1 Scope guard (`scope.py`) - plain code, default deny

- Target matching: exact host, IP-in-CIDR, URL host extraction.
- A **CIDR target must be fully inside** an authorized network. Exclusions always win for single hosts; for CIDRs, overlapping exclusions are passed to nmap as `--exclude` .
- Optional allowed ports, allowed tools, time window.
- Four intensity classes: `passive < standard < intrusive < exploit` . `intrusive` and above set `needs_approval` . `exploit` is rejected unless scope max allows it.
- `enforce()` raises `ScopeViolation` ; the engine logs `action.blocked` and never sends traffic.

8.2 Evidence and audit (`runner.py` , `store.py`)

- Every tool output is saved under `data/artifacts/<assessment>/` and hashed (SHA-256). An `Evidence` object stores tool, mode (live/import), file path, hash, locator, excerpt.
- The UI/API can re-read the raw file and re-verify the hash (`X-Evidence-Hash-Verified` header).
- **Audit chain:** each row stores `hash = SHA256(prev_hash | assessment | ts | actor | action | detail)` . `verify_audit()` recomputes the whole chain; a test proves tampering is detected. Chain status is shown in the UI header and PDF.

- Jobs: timeout (default 300s), one retry for transient errors, `ToolUnavailable` is not retried and becomes a coverage gap.

8.3 Correlation and the Confidence Ledger (`correlation.py` , `reference.py`)

- **Normalize:** each finding title is matched against ordered regexes into a canonical `issue_key` (clickjacking, missing-csp, xss, sqli, directory-listing, cookie-httponly ...), gets a canonical CWE, OWASP category, remediation text and MITRE ATT&CK ids.
- **Merge key:** `host | port | issue_key` (plus URL path and parameter for per-URL classes: xss, sqli, path-traversal, csrf, info-disclosure-file, default-credentials). Different hosts/ports are never merged. All occurrences, evidence ids, CVEs, URLs and the highest severity/CVSS are kept.
- **Confidence points (deterministic, each with a written reason):**
- +40 tool test/rule, or +25 version-only (and +15 if the version is in the reference table), or +50 directly observed open service
- +25 for a second independent tool, +10 for each further tool
- +15 request/response proof (Burp, or "Evidence:" in the description)
- -5 import-only, -30 distro package (version-only on debian/ubuntu/el7/el8/rhel)
- Validated findings have a floor of 90; refuted findings are capped at 10.
- **Status ladder:** observed -> suspected (version-only) -> corroborated (2+ tools) -> validated; or refuted (false positive, reason kept) or fixed (after retest).

8.4 Safe validation (`adapters/validators.py`)

One `curl -i` request (from the tools container, no payloads, no approval needed) decides **confirmed / refuted / inconclusive**. Examples: HSTS on a plain-HTTP site is *refuted* (browsers ignore it); clickjacking is refuted if X-Frame-Options or CSP frame-ancestors is present; cookie flags are checked against `Set-Cookie` ; directory listing checks for an "Index of /" title. This is how ARGUS removes false positives.

8.5 Risk ranking (`risk.py`)

```
priority = min(10, base x exposure x exploitability x asset_criticality x confidence_weight +
```

- **base:** the tool's CVSS if any, otherwise the midpoint of the tool-reported severity (critical 9.5, high 8.0, medium 5.5, low 2.5, info 0.5), labelled "no CVSS".
- **exposure:** 1.0 (assumed reachable without auth).
- **exploitability:** 1.3 if the CVE is in the *partial offline* CISA KEV snapshot, else 1.0.
- **criticality:** 1.0 default.
- **confidence weight:** validated 1.0, corroborated 0.95, observed 0.85, suspected 0.75.
- **path bonus:** +0.5 for findings that are part of a hypothesised attack chain.
- Refuted findings are excluded (priority 0). Bands: >=9 critical, >=7 high, >=4 medium, >=1 low.
- No LLM ever produces a number. The UI shows every factor and its source.

8.6 Attack-path graph (`graph.py`)

Nodes: host -> service -> finding -> hypothesis. **Solid green edges** are backed by evidence; **dashed orange** edges are rule-based hypotheses, e.g. XSS + HttpOnly-less cookie = session hijack; XSS + no CSP; clickjacking + CSRF; known-CVE = possible remote compromise; cleartext protocol = credential interception.

8.7 PDF report (`report.py`)

Cover (scope, exclusions, authorized by, window, audit-chain status) -> 1 Executive summary -> 2 Methodology (tool run table, truthful integration matrix) -> 3 Severity distribution and prioritized table -> 4 Detailed findings (each splits **OBSERVED tool evidence** from **ANALYSIS rule-based inference**) -> 5 Impact and remediation -> 6 Limitations, assumptions, human approvals -> Appendices: A evidence index with SHA-256, B refuted false positives, C agent decision log, E informational observations, D hash-chained audit trail.

---

10. Web UI (what the user sees)

Left panel: request box, presets ("Web app check", "One server, safe only", "Juice Shop"), **Check scope & make a plan**, a **safety demo** button that asks for `8.8.8.8` and `scanme.nmap.org` (rejected, no traffic), the allowed-scope box, past assessments.

Workspace: status bar, 6-step stepper (Scope, Plan, Scan, Approve, Review, Report), a "what to do next" banner, and tabs: **Overview** (tiles, severity donut, safety summary, top risks) / **Plan & reasoning** (scope check, Decision Cards, hypotheses) / **Live activity** (tool runs, event stream) / **Findings** (filter, search, import, detail with OBSERVED vs ANALYSIS, Retest, request Metasploit check) / **Attack paths** / **Approvals** / **Ask ARGUS** / **Report & audit** / **Tools** (integration matrix).

Live updates use Server-Sent Events. Deep link: `/?a=<assessment id>&tab=findings` .

---

11. API reference (all under `/api`)

Method + path	Purpose
`GET /health`	ok, planner mode, docker availability, audit-chain status
`GET /matrix`	tool integration status, probed live
`GET /scope/default` , `POST /scope/check`	view default scope; dry-run a scope decision
`POST /assessments` , `GET /assessments` , `GET /assessments/{id}`	create (+plan), list, full state
`POST /assessments/{id}/approve-plan`	start the investigation loop (409 if not planned/stopped)
`POST /assessments/{id}/stop`	kill switch

Method + path	Purpose
`POST /assessments/{id}/import` , `/import-sample/{tool}`	import Nessus/Burp/OpenVAS
`POST /assessments/{id}/approvals/{ap}/decide` , `/retry`	decide or re-request approval
`POST /assessments/{id}/findings/{fid}/propose-validation`	request Metasploit read-only check (approval-gated)
`POST /assessments/{id}/findings/{fid}/retest`	safe retest
`POST /assessments/{id}/ask`	copilot Q&A
`GET /assessments/{id}/graph`	attack-path graph
`GET /assessments/{id}/evidence/{ev}/raw`	raw evidence with hash verification header
`GET /assessments/{id}/audit`	audit entries + chain verdict
`GET /assessments/{id}/report.pdf`	the PDF
`GET /assessments/{id}/events`	SSE stream

---

12. How to run it

```
 pip install -r requirements.txt
.\scripts\start.ps1      # docker compose up -d --build (lab), then uvicorn on http://localhost
.\scripts\stop.ps1       # removes only the ARGUS containers
cd backend; python -m pytest -q          # unit tests
python scripts\e2e_live.py               # live E2E (needs lab + server running)
python scripts\make_test_report.py       # builds docs/ARGUS_Test_Report.pdf from results
```

Requirements: Windows 10/11 (Linux/macOS work too), Docker Desktop with ~6 GB free RAM, Python 3.11+. First run pulls images and builds the tools image (several minutes). No API key needed. Runtime data lives in `data/` (git-ignored).

---

13. Plan vs. what was actually built (important for honesty in the demo)

Original plan (PDF)	What exists in the code	Status
React + Vite + Tailwind + Cytoscape	Vanilla JS, hand-made SVG graph	Deviation (documented)

Original plan (PDF)	What exists in the code	Status
WeasyPrint + Jinja2 PDF	reportlab	Deviation (documented)
SQLite + SQLModel	Plain sqlite3 with JSON blobs	Minor deviation
Claude drives planning via tool calling	Rule planner is the default; optional Gemini (preferred, verified live) or Claude (unit-tested only) ranks code-made candidates, never invents one	Deviation, Gemini path verified live
OpenVAS live via GMP	Import-only	Deviation (documented)
Metasploit via msfrpcd + RPC	`docker run ... msfconsole -x` with allow-listed aux modules	Different mechanism
nmap `vulners` NSE	Not used; small hand-written version->CVE table	Deviation
Scope: whole lab subnet	Subnet minus .1/.2/.3	Fixed bug (gateway = dev PC)
Burp Pro shown as "Unsupported"	Shown as Import-only (Pro REST API "Planned")	Wording differs
Grounding Validator rejects AI text	**Now wired in**: copilot answers, the optional Claude rationale (discarded on failure) and the report narrative are checked; the PDF states the result	Fixed
Secrets redacted before sending to LLM	The LLM summary is limited (targets, `host:port service product version`, finding labels, hypotheses) but there is no explicit redaction step	Fixed: `planner.redact` strips credentials/tokens before the Claude call
Exposure 0.85/0.7, exploit 1.15, analyst-set asset criticality	Exposure fixed at 1.0, KEV 1.3; analyst can now set criticality 0.8/1.0/1.2 per host (API + Findings detail)	Partly fixed
Confidence rule "-25 another tool tested and did not reproduce"	Not implemented	Missing
Confidence level labels Low/Medium/High	Shown in UI and PDF (<40 low, 40-69 medium, >=70 high)	Fixed
"REPLAY" demo mode	Not implemented	Missing (not needed if live)
Retest "diff" view	Each retest records before/after status, verdict and the specific reason (e.g. "X-Frame-Options header is absent" -> "X-Frame-Options: DENY is present"), building a readable	Fixed (covers validatable classes; no raw-response line-by-line diff)

Original plan (PDF)	What exists in the code	Status
	change history over multiple retests. Shown in both the UI and the PDF report	
Redirect/out-of-scope link dropping from crawlers	All tool results are now filtered by scope; discarded hosts are logged as limitations and audited	Fixed
ZAP active scan actually reaching DVWA's vulnerable pages	ZAP never fills in a login form on its own, so an unauthenticated scan only ever found `login.php`. `lab_init` now logs in to DVWA and registers the session with ZAP's `httpSessions` API (a Replacer rule looked right but only replaces a Cookie header that already exists - it never adds one, so it silently did nothing on a fresh ZAP daemon). The active scan is also restricted to the fast SQLi/XSS scanners (the *time-based* SQLi variants were dropped - they wait on deliberate response delays and intermittently ate the whole 4-minute budget)	Fixed (lab-setup + `zap.py` fix, found and corrected through live testing, including reverting one attempted fix that made things worse); verified live - real SQL Injection/XSS findings from DVWA, 99/100 e2e checks

---

13a. Compliance against the official DataOps brief (PS-1) and judging rubric

This section maps directly to the organizers' actual problem statement and the 100-point rubric (not the earlier internal plan PDF above), so it's the one to check before the demo.

Required deliverables (brief section 9):

Deliverable	Status
9.1 Working AI agent demoed against an organizer-authorized lab target	Working against our own Docker lab; needs pointing at whatever target the organizers give at demo time (just a scope change, no code change)
9.2 Architecture overview (AI models, orchestration, adapters, data flow, state/memory, security boundaries)	`docs/ARCHITECTURE.md` - all six named items present
9.3 Sample AI-generated, evidence-backed VAPT report (PDF)	`docs/ARGUS_Sample_VAPT_Report.pdf` - from a real run with real SQLi/XSS findings
9.4 Source repository (setup, dependencies, licensing, execution instructions)	`README.md` + this doc
9.5 Human-approval demonstration with a decision/audit trace	Approvals tab + `/api/assessments/{id}/audit`; live-tested

Deliverable	Status
9.6 Integration status matrix (limitations, assumptions, unsupported scenarios)	`docs/INTEGRATIONS.md` , also live at the UI's Tools tab

Judging criteria (100 pts) - where each one is demonstrated:

Criterion	Pts	Where to show it
End-to-End Coverage	15	Request -> plan -> approve -> scans run -> findings -> risk ranking -> report, in one assessment
AI Reasoning & Decisions	20	Decision Cards (every step's rationale, even without an LLM); with a Gemini/Claude key, the model's own rationale appears and is grounding-checked
Technical Accuracy	15	Confidence levels, validation status, evidence links on every finding
Correlation & False Positives	15	Cross-tool merge demo (clickjacking from 3 tools -> 1 finding); HSTS-on-HTTP auto-refuted by a real request
Risk Prioritization	10	Priority formula with every factor shown and sourced, never a bare number
Safety & Scope Control	10	Out-of-scope target refused live; intrusive actions queued for approval; audit chain
Report Quality	10	PDF report - all 6 required sections (brief 7.1-7.6) present and verified by the E2E suite
Efficiency & Robustness	5	Restart recovery, partial-result handling, ZAP-killed-mid-scan recovery, clean error on bad input

Judges' live-demo checklist (brief section 10) - rehearsal order for the video/demo: 1. Enter a request + target, show scope validation (try an out-of-scope IP, watch it get refused). 2. Show the generated plan with tool selections and reasoning (Decision Cards). 3. Approve the plan, watch real tools execute (Live activity tab) - and import a sample Nessus/Burp/OpenVAS file to show import-only honestly. 4. Open Findings: show a merged cross-tool finding and a refuted false positive. 5. Show risk ranking (the formula, not a bare score) and approve an intrusive ZAP active scan or Metasploit check live. 6. Download the PDF report, click through to a finding, trace it back to its raw evidence (SHA-256 verified). 7. Kill a tool mid-scan (or point out the ZAP-down resilience test) and show the assessment still completes; name the two import-only tools honestly.

---

14. Strengths to emphasise at the demo

1. **Scope guard in code** - the out-of-scope demo (`8.8.8.8`) sends zero traffic and is logged. The `.1` gateway exclusion is a real incident turned into a safeguard.
2. **Decision Cards** with options considered, not a fixed script - real reasoning, not a fixed pipeline, satisfied without any LLM at all.
3. **False-positive removal that actually happens** (HSTS on HTTP refuted by a real request).

4. **Cross-tool merge**: 3 independent reports of clickjacking become 1 finding with all evidence and confidence 75 (tested).
5. **Evidence you can click**: raw output + SHA-256 + live hash re-verification.
6. **Tamper-evident audit chain** and a kill switch.
7. **Honest integration matrix** and an AI disclosure document.
8. **Robustness**: restart recovery, partial ZAP results, clean 504 on retest timeout, coverage-gap reporting, ZAP-killed-mid-scan recovery.
9. **A real injection finding, not a staged one**: the live ZAP active scan (after the DVWA auth fix) genuinely finds SQL Injection and reflected XSS on DVWA from scratch, with the request/response evidence saved and hashed - this can be re-run live in front of judges.
10. **Debugged with real evidence, not guesswork**: the DVWA active-scan fix is a good "how we actually debug" story for Q&A - traced with ZAP's own `sendRequest` API to prove the auth cookie genuinely wasn't being sent, not assumed.
11. **Genuinely adaptive, not just rule-following, with the LLM enabled**: with Gemini active, the agent chose to fully investigate one host (recon -> enumeration -> analysis -> triage) before starting the next, instead of the rule planner's breadth-first "every host's recon, then every host's enumeration..." order. That's real adaptive, context-driven sequencing - directly what the brief's "AI Reasoning & Decisions" (20/100 pts, the single biggest category) and the "Multi-step adaptive replanning" bonus feature ask for, and it only shows up once a real model is choosing.

15. Known weaknesses / risks

- The Gemini planner path is **verified live** (real assessment, real contextual rationale, confirmed in the decision log). The Claude path remains unit-tested only (no Anthropic key available).
- `lab_scope()` is hard-coded; the organizer's real lab must be set in `api.py` or sent as a `scope` object.
- PDF fonts assume Windows paths (falls back to Helvetica elsewhere).
- KEV snapshot and version->CVE table are tiny; most CVE hints will only appear for the few listed versions.
- Unit tests: 34 pass. The live Docker E2E suite (`scripts/e2e_live.py` , ~100 checks against the real lab) usually runs 96-99/97-100; the recurring gap, when it happens, is the DVWA ZAP-session flakiness below, or (separately) a timing race in the kill-switch test (a fast nmap job finished before the test's 12s sleep elapsed, so there was nothing left to kill) - neither is a safety or correctness issue.
- A few plan items remain unimplemented by design choice, not oversight: a "-25 if another tool re-tested and didn't reproduce" confidence rule, and exposure factors other than 1.0. None are required by the official brief.
- **DVWA/ZAP authenticated-session flakiness (known, partially mitigated).** The active-scan-finds-nothing bug (section 13 below) was fixed for the common case, but during a long run with many concurrent ZAP jobs across two hosts, the authenticated session ZAP uses for DVWA can still occasionally stop working mid-run - plausibly ZAP's own "track session automatically" behaviour (it updates its stored cookie from *any* Set-Cookie it observes, which can race with DVWA regenerating its session under concurrent load) outliving our control over it. `lab_init` now spiders DVWA and verifies `/vulnerabilities/sqli` is actually reachable before finishing, retrying the whole login+session sequence up to 3 times - this reliably fixes it at setup time (proven: attempt 1 failed, attempt 2 succeeded, in testing) but doesn't guarantee it stays fixed through a very long multi-host run afterward. **For the demo:** re-run `scripts/lab_init.ps1` right before the segment that shows the live SQLi finding, and keep that run focused on DVWA rather than a long two-host sweep - this is the scenario it's been verified reliable in. If it's ever not reproduced live, the sample report (`docs/ARGUS_Sample_VAPT_Report.pdf`) already contains a real one from a verified run.